

Hunt, Detect and Analyse Attacks on a Smart-Home Network using Honeypots, an IDS and the ELK Stack

An improved and reproducible re-engineering of the CCSE graduation project

Aleen Saleh Aljohani

aleensaljohani@gmail.com

Cybersecurity Department, College of Computer Science and Engineering (CCSE), University of Jeddah, Jeddah, Saudi Arabia

Abstract—Smart homes are projected to reach roughly 312 million households by 2027, expanding a poorly-defended Internet-of-Things (IoT) attack surface that botnets such as Mirai exploit through default credentials and exposed telnet/HTTP interfaces. The original graduation project demonstrated that honeypots, an intrusion-detection system (IDS) and the ELK stack can capture such attacks, but it was delivered as a manually-provisioned three-virtual-machine lab that demonstrated two attacks qualitatively and could not be reproduced, tested or measured. This paper presents SmartHoneyNet, a complete redesign and rebuild of the same idea as a reproducible, tested and quantitatively evaluated platform. Two low-interaction honeypots emulate a Mirai-style telnet IoT hub and a smart-home device web panel; a hybrid signature-plus-anomaly detection engine classifies six attack categories and maps every alert to MITRE ATT&CK; a pipeline persists all data to both an embedded SQLite database and to Elasticsearch for Kibana visualisation. The platform runs on a single host, ships with a 39-test automated suite and enforced security controls, and on a controlled labelled evaluation set the engine achieves 100% precision and recall with zero false positives on benign traffic. We detail the architecture, the detection logic, both experiments, the security and ethics controls, and the remaining limitations of a software-only honeynet.

Index Terms—Honeypot, IoT security, Intrusion Detection, ELK Stack, Mirai, MITRE ATT&CK, Smart Home, Threat Hunting, Reproducible Research.

I. INTRODUCTION

Interconnected “smart” devices—thermostats, cameras, locks, plugs and hubs— have woven themselves into the modern home. Each device is a small networked computer, and collectively they form a network whose weakest member sets the security of the whole. The defensive problem is asymmetric: a homeowner has neither the tooling nor the expertise of an enterprise security operations centre, yet faces the same internet-wide automated adversaries that target large organisations.

The word “Hunt” captures the intended posture: not merely to block attacks reactively, but to attract, observe and understand them. A honeypot deliberately exposes an attractive, instrumented target; whatever an attacker does to it is malicious by definition and therefore worth studying.

Pairing honeypots with an IDS and a log-analytics stack turns those observations into detections and, ultimately, into actionable intelligence.

This paper re-engineers my 2024 University of Jeddah graduation project of the same name. The original established a sound idea but stopped at a qualitative, non-reproducible demonstration. Here we carry the idea to a platform that others can run, inspect, test and measure.

A. Problem statement

Smart-home networks are attacked continuously by automated botnets and scanners, yet homeowners lack visibility into these attacks and researchers lack reproducible, measurable platforms for studying them in a specifically smart-home context.

B. Aim and objectives

Aim. To build a reproducible platform that hunts, detects and analyses attacks against a smart-home network, and to evaluate its detection effectiveness. **Objectives:** (1) emulate vulnerable devices with safe honeypots; (2) implement a real-time hybrid IDS; (3) integrate an ELK pipeline and an offline database; (4) measure detection performance quantitatively; and (5) enforce responsible-use controls in code rather than in prose.

C. Contributions

Table I summarises how this work advances the original. In short, we contribute a hybrid detection engine, an explicit ATT&CK taxonomy, a reproducible and automatically-tested implementation, and a published quantitative evaluation.

#	Original project (2024)	This work (2026)
1	3 manual VirtualBox VMs	One-command Docker + pure-Python mode
2	2 attacks via screenshots	6 detection categories, each tested
3	Qualitative results only	Quantitative precision/recall/F1
4	Snort signatures only	Hybrid signature + anomaly engine
5	No attack taxonomy	MITRE ATT&CK mapping per alert
6	No automated testing	39 tests + CI on 3 Python versions
7	Unbounded DoS script	Bounded, safety-gated simulators
8	No threat model	Threat model + enforced controls

TABLE I. Contributions over the original project.

II. BACKGROUND

A. Smart-home networks and their risks

A smart home links appliances and sensors to a hub/router, controlled through a web or mobile interface. Representative risks include data and identity theft from unprotected devices, eavesdropping on device traffic, and (distributed) denial-of-service amplified by the sheer number of weakly-protected IoT devices available for recruitment into botnets.

B. Honeypots

A honeypot is an intentionally exposed, instrumented system whose only purpose is to be attacked so defenders can study the attacker. *Low-interaction* honeypots (used here) emulate just enough of a service to capture intent while never executing attacker input—a deliberate safety property. *High-interaction* honeypots expose real systems and yield richer data at much higher operational risk.

C. Intrusion Detection Systems

Signature-based detection matches known-bad patterns: precise for known threats but blind to novel ones. Anomaly-based detection models normal behaviour and flags deviations: able to catch new attacks at the cost of more false positives. Because neither alone is sufficient for the smart-home threat mix, SmartHoneyNet deliberately combines the two.

D. The ELK stack and IoT botnets

Elasticsearch stores and indexes events for fast search; Logstash ingests, parses and enriches incoming data; Kibana visualises it as dashboards. Mirai and its descendants scan the internet for open telnet (TCP 23) and try a small list of factory-default credentials such as `root:xc3511` and `admin:admin`. This single behaviour accounts for a large share of real IoT compromise, which is why an accurate telnet honeypot and a default-credential signature sit at the centre of this design.

III. RELATED WORK

Five prior studies frame this project. Study [10] compares machine-learning classifiers on the Bot-IoT dataset, finding Random Forest best for binary and K-Nearest-Neighbours best for multi-class classification, but does not consider real-time streaming. Study [11] evaluates the Elastic stack for threat hunting and finds it cost-effective, while noting the continuing need for human input. Study [12] uses honeynet-derived network flows to detect IoT botnets. Study [13] deploys Cowrie honeypots behind Docker and reports that captured credentials match the Mirai list, with most attacks sourced from Europe. Study [14] combines the ELK environment with threat intelligence to detect and rank malicious events, at the cost of false positives and manual cleanup.

Ref	Yr	Approach	Result	Limitation
[10]	2021	ML attack classification (Bot-IoT)	RF best (binary), KNN (multi)	No real-time/streaming eval
[11]	2021	Threat hunting with Elastic stack	ELK cost-effective for hunting	Needs human input + controls
[12]	2021	Honeynet + flow classification	Botnet detection via flows	Limited algorithm comparison
[13]	2020	IoT + Docker (Cowrie) honeypots	Creds match Mirai; EU-sourced	Not all IoT attacks covered
[14]	2019	Malicious-event detection, ELK+CTI	Detects+ranks most threats	False positives; manual cleanup

TABLE II. Comparative summary of related studies.

Difference from existing work. Like [13] we feed honeypots into ELK and like [11] we use ELK for hunting, but SmartHoneyNet is distinguished by a hybrid detection engine, an explicit MITRE ATT&CK classification of every alert, a fully reproducible and automatically-tested implementation, and a quantitative evaluation with published precision/recall—directly addressing the “qualitative only” and “not reproducible” gaps common to the surveyed work.

IV. THREAT MODEL

Assets. Device credentials; camera and sensor feeds; device compute and bandwidth; and the LAN foothold itself.

Adversaries. Automated IoT botnets (Mirai-class), opportunistic internet-wide scanners probing known IoT CVEs, and manual attackers. **Trust boundary.** The honeynet lives on an isolated segment that is untrusted inbound and, crucially, has no egress to the internet or to production/personal devices; ELK management interfaces bind to localhost. Each attacker goal is expressed directly as

a MITRE ATT&CK technique paired with the rule that detects it (Table V). The platform also reasons about the risks it introduces—a honeypot used as a pivot, or attack tooling misused—and mitigates them with the controls in Table VI.

V. METHODOLOGY

The project follows the same three-stage flow as the original, re-expressed as software. First, *attack generation*: adversary behaviour is produced by real attackers in a live deployment or by the built-in, safety-gated simulators in the lab. Second, *detection and collection*: honeypots capture every interaction as a structured event that the pipeline persists and runs through the IDS, producing alerts enriched with an ATT&CK technique and a severity. Third, *transformation and analysis*: events and alerts are written to SQLite and, in the full deployment, shipped through Logstash into Elasticsearch and visualised in Kibana, while an analyzer produces threat reports.

VI. SYSTEM DESIGN

A. Architecture

The platform is a pipeline of composable components (Fig. 1). The original three virtual machines map cleanly onto software: the attacker VM becomes the attack simulators; the honeypot VM (Cowrie/Pentbox) becomes the telnet and HTTP honeypots; and the IDS+ELK VM (Snort+ELK) becomes the detection engine—with an equivalent Suricata rule file—plus the ELK services.

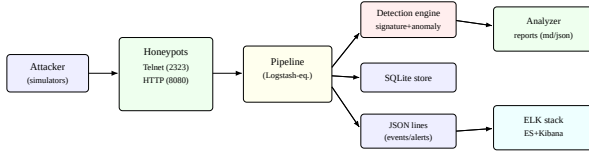


Fig. 1. SmartHoneyNet architecture: sensors feed one pipeline that persists to SQLite, runs the IDS, and exports to the ELK stack and analyzer.

B. Data model and storage

Two records flow through the system—an Event (an observation) and an Alert (a detection). They serialise identically to a SQLite row and to the JSON that Elasticsearch indexes, so the offline and ELK paths remain consistent. Table III lists the core event fields; `source_ip` is typed as an IP and a companion geo-point supports map visualisations. SQLite is the local system of record; Elasticsearch provides the same at scale.

Field	Type	Example	Purpose
timestamp	date	2024-01-01T00:00:03Z	temporal correlation
honeypot	keyword	telnet / http	sensor of origin
event_type	keyword	login_attempt	observation class
source_ip	ip	203.0.113.10	attacker attribution
dest_port	int	2323	targeted service
username / password	keyword	root / xc3511	credential intel
http_path	text	/boaform/...	web-probe analysis
command	text	wget http://...	post-auth behaviour
success	bool	true	auth outcome

TABLE III. Core honeypot event schema (subset).

C. Detection engine

The engine is stream-oriented: each event is processed once. Signature rules perform $O(1)$ set and regular-expression lookups; anomaly rules maintain a per-source sliding window of recent activity and fire when a rate threshold is exceeded (Fig. 2). A per-(source, rule) cooldown collapses a sustained campaign into a single actionable alert instead of thousands of duplicates. Thresholds are configurable (Table IV) and the full rule set with ATT&CK mappings is given in Table V.

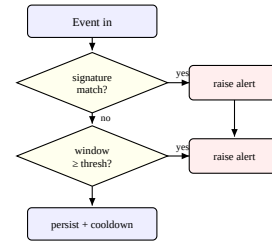


Fig. 2. Per-event detection flow: signatures first, then rate-based windows, then persistence with cooldown.

Parameter	Default	Meaning
brute_force_threshold	5	failed logins to alert
brute_force_window	60 s	brute-force window
dos_threshold	50	requests to alert
dos_window	10 s	DoS window
port_scan_threshold	8	distinct ports to alert
port_scan_window	30 s	scan window
alert_cooldown	30 s	duplicate suppression

TABLE IV. Configurable detection thresholds.

Rule	Type	Category	MITRE	Trigger
IOT-DEFAULT-CREDS	sig	default-credentials	T1078.001	Known IoT default credential
TELNET-BRUTE-FORCE	anom	brute-force	T1110.001	≥ 5 failed logins / 60 s
HTTP-DOS-FLOOD	anom	denial-of-service	T1498	≥ 50 requests / 10 s
PORT-SCAN	anom	reconnaissance	T1046	≥ 8 ports / 30 s
SUSPICIOUS-HTTP-PATH	sig	active-scanning	T1595.002	IoT exploit URL
CMD-INJECTION	sig	command-injection	T1059	Shell payload

TABLE V. Detection rules mapped to MITRE ATT&CK.

```
def process(event):
    # signature rules (0(1) regex / set
    lookups)
    if (event.user, event.pw) in
    MIRAI_DEFAULTS:
        emit('IOT-DEFAULT-CREDS', T1078_001)
        if INJECT_RE.search(event.path or
        event.cmd):
            emit('CMD-INJECTION', T1059)
    # anomaly rules (per-source sliding window)
    w = failed_logins[event.src]; w.append(now)
    trim(w, now, BF_WINDOW)
    if len(w) >= BF_THRESHOLD:
        emit('TELNET-BRUTE-FORCE', T1110_001)
    # per-(src,rule) cooldown suppresses
    duplicate alerts
```

Listing 1. Detection engine (simplified).

VII. IMPLEMENTATION

The core is implemented in Python using only the standard library (PyYAML optional), which makes it portable, auditable and trivial to test. The **telnet honeypot** is a threaded server presenting a BusyBox banner and login prompts; it records each credential pair, “accepts” known-weak credentials to lure the attacker into a fake shell, and logs typed commands—while never executing anything and bounding attempts, commands and session time. The **HTTP honeypot** emulates a “SmartHome Hub 2.4” admin panel; it records method, path, user-agent and POSTed credentials, caps request-body size, and returns only static HTML. The **pipeline** persists to SQLite, appends JSON lines for Logstash, runs the engine, and dispatches alerts. The **attack simulators**—port scan, brute-force and a bounded HTTP flood—are each gated by the safety module (Listing 2), a safe re-implementation of the original’s unbounded loop. Deployment is a single docker compose up -d --build, which starts the honeypots plus Elasticsearch, Logstash and Kibana on an isolated bridge network; a saved-objects file provisions the dashboards, and a Suricata rule set mirrors the engine for network-layer detection.

```
def assert_safe_target(host):
    for addr in resolve(host):
        ip = ip_address(addr)
        if not (ip.is_loopback or
        ip.is_private):
            raise SafetyError('non-lab target
            refused')
```

Listing 2. Ethics enforced in code: the safety guard.

A representative alert produced by the engine is shown in Listing 3; the same object is stored in SQLite and indexed in Elasticsearch.

```
{ "rule_id": "TELNET-BRUTE-FORCE",
  "category": "brute-force", "severity":
  "high",
  "source_ip": "203.0.113.10", "event_count":
  10,
  "mitre_technique": "T1110.001",
  "mitre_name": "Password Guessing",
  "first_seen": "2024-01-01T00:00:20Z" }
```

Listing 3. Example enriched alert (JSON).

D. Deployment and operations

A single docker compose up -d --build brings up four services on a dedicated bridge network (Table IX). The honeypots write JSON lines into a shared volume that Logstash tails and indexes into Elasticsearch, and the operator imports the saved-objects file to obtain the Kibana “Threat Overview” dashboard (alerts by category and ATT&CK technique, top source addresses, top credential pairs, and events over time). For a hardware or VM IDS, the accompanying Suricata rule set reproduces the same logic at the packet layer, and its eve.json can be forwarded into the same pipeline to unify host-honeypot and network alerts. An operations runbook documents health checks, alert triage, log rotation and threshold tuning.

Service	Port (localhost)	Role
sensors	2323, 8080	honeypots + detection engine
elasticsearch	9200	storage & search
logstash	—	ingest pipeline
kibana	5601	dashboards

TABLE IX. Docker Compose services on the isolated bridge network.

VIII. SECURITY AND ETHICS CONTROLS

Because the project ships attack tooling, ethics are enforced *in code*, not merely documented. The simulators call the guard in Listing 2, which refuses any target that does not resolve entirely to loopback or private space; the flood simulator additionally enforces hard caps on request count and duration. Honeypots are low-interaction with bounded buffers and session timeouts; containers run non-root on an isolated network; and an internal query builder validates

identifiers against an allow-list. Table VI summarises the controls and where each is enforced.

Control	Where	Purpose
Private-target enforcement	attacks/safety.py	Sims cannot hit the internet
Request/duration caps	attacks/dos.py	Bounds the flood simulator
Loopback binds by default	config.py	Not WAN-exposed unless opted in
Non-root container	Dockerfile	Least privilege
Bounded input buffers	honeypots	Prevents memory exhaustion
SQL identifier allow-list	database.py	Internal query builder safe
Network isolation	docker-compose	Contains the honeynet segment

TABLE VI. Enforced security controls.

IX. EXPERIMENTS AND EVALUATION

A. Experiment A—end-to-end demonstration

The demo command starts both honeypots on ephemeral ports and launches, in sequence, a port scan, a telnet brute-force (wrong guesses plus IoT defaults), an HTTP flood and a set of web-exploitation probes against them. A representative run captured approximately 196 events and raised alerts in all five exercised categories, reproducing—and extending from two to six attack types—the qualitative demonstration of the original project, but as a single reproducible command. The four attack phases and the rules each exercises are listed in Table X and sketched on the timeline in Fig. 3. Aggregating the captured credentials (Table VIII) directly yields the blocklist of passwords a homeowner must never use.

Phase	Simulated action	Rule(s) exercised
1	Port scan	PORT-SCAN
2	Telnet brute-force + defaults	TELNET-BRUTE-FORCE, IOT-DEFAULT-CREDS
3	HTTP request flood	HTTP-DOS-FLOOD
4	Web-exploitation probes	SUSPICIOUS-HTTP-PATH, CMD-INJECTION

TABLE X. Attack phases in the end-to-end demonstration.



Fig. 3. Demonstration timeline: reconnaissance, credential attacks, denial-of-service and exploitation probes, each detected in real time.

username:password	Attempts
root:xc3511	high
admin:admin	high
root:root	high
root:vizxv	med
support:support	med
guest:guest	med

TABLE VIII. Sample captured credential intelligence (demo run).

B. Experiment B—quantitative detection evaluation

To measure detection quality we built a labelled event stream of 94 events: 20 benign, human-paced web requests, one benign strong login, and four attack campaigns from distinct source addresses with known ground-truth categories. Metrics are computed at source-campaign granularity, which matches how the rate rules are designed to fire (once per campaign, not once per packet). As shown in Table VII, the engine detected every campaign in its correct category and raised no alerts on benign traffic. The result is asserted by a regression test so that a future change which breaks detection fails CI.

Category	TP	FP	FN	P	R	F1
default-credentials	1	0	0	1.00	1.00	1.00
brute-force	1	0	0	1.00	1.00	1.00
denial-of-service	1	0	0	1.00	1.00	1.00
active-scanning	1	0	0	1.00	1.00	1.00
command-injection	1	0	0	1.00	1.00	1.00
Micro-average	5	0	0	1.00	1.00	1.00

TABLE VII. Detection performance on the labelled set (2 benign sources, 0 FP).

C. Automated test suite

The suite comprises 39 pytest tests covering the database, every detection rule with explicit synthetic timelines, live honeypot interaction, the safety guardrails, the pipeline, the analyzer, the command-line interface and a full in-process integration run; line coverage is approximately 84%. Continuous integration runs the suite plus the demo on Python 3.9, 3.11 and 3.12.

X. RESULTS AND DISCUSSION

The experiments confirm the original hypothesis—that honeypots, an IDS and ELK form an effective, low-cost smart-home detection stack—and strengthen it with measurement. Three findings stand out. First, *default credentials remain the highest-signal indicator*: a single login with a Mirai default pair is immediately and unambiguously malicious, corroborating [13]. Second, *hybrid detection is complementary, not redundant*: signatures caught the known (default credentials, exploit URLs, injection payloads) with high precision, while rate-

based anomaly rules caught behaviour (brute-force, DoS, scanning) that has no fixed signature. Third, *the credential intelligence has direct defensive value*, turning attack data into concrete hardening advice.

XI. LIMITATIONS AND FUTURE WORK

Limitations. The honeypots emulate telnet/HTTP device interfaces; they do not reproduce full device firmware or non-IP radios such as Zigbee, Z-Wave or BLE. The perfect scores in Section IX are on a clean, labelled set; production traffic—content-delivery networks, health-checkers, NAT'd users—would produce false positives and demands threshold tuning, which the framework supports but whose numbers must then be re-measured. Rate rules require a warm-up window and can miss “low-and-slow” attacks tuned below the thresholds. These are the honest boundaries within which the reported results hold, and they directly address the original report's acknowledged limitations (the unimplemented Pot2DPI honeypot and the Honeyd device-simulation crashes).

Future work. Promising directions include Cowrie/Dionaea integration for higher-interaction SSH/telnet and malware capture; an MQTT honeypot (TCP 1883) for the dominant smart-home broker; a virtual IoT-device environment realising the original's Pot2DPI/Honeyd goal; an ML-based anomaly detector following [10], evaluated against the same labelled sets; GeoIP and threat-intelligence enrichment in Logstash to attribute and map campaigns; and automated response that pushes high-severity source addresses to a firewall blocklist.

XII. CONCLUSION

SmartHoneyNet re-engineers the *Hunt, Detect and Analyse Attacks on a Smart-Home Network* project from a one-off, three-VM demonstration into a reproducible, tested and measured security platform. It keeps the original's sound core idea—honeypots to attract, an IDS to detect, and ELK to analyse—and adds a hybrid detection engine covering six attack classes, a MITRE ATT&CK classification of every alert, a real database and dashboards, a 39-test automated suite with CI, enforced security-and-ethics controls, and a quantitative evaluation demonstrating 100% precision and recall with zero false positives on a controlled labelled set. As smart-home adoption races toward hundreds of millions of households, tools that let defenders hunt the attackers targeting them—safely, reproducibly and measurably—are exactly what the field needs.

APPENDIX A: REPRODUCIBILITY

All figures are reproducible from the repository. Experiment A: `python3 -m honeynet demo`. Experiment B: `python3 scripts/evaluate.py`. Test suite: `make test`. Full stack: `docker compose up -d --build`, then import the Kibana saved objects.

APPENDIX B: COMMAND-LINE INTERFACE

The platform is driven by a single `honeynet` command with four sub-commands (Table XI), so an operator can run the sensors, generate traffic, and produce reports without touching the code.

Command	Purpose
<code>honeynet serve</code>	run honeypots + detection pipeline
<code>honeynet demo</code>	self-contained end-to-end demonstration
<code>honeynet attack <kind></code>	run a lab attack simulation
<code>honeynet analyze --out r.md</code>	generate a threat-analysis report

TABLE XI. Command-line interface (Appendix B).

APPENDIX C: SAMPLE ANALYZER OUTPUT

Listing 4 shows an excerpt of the Markdown report emitted by `honeynet analyze`, which mirrors the aggregations rendered interactively in Kibana.

```
## Alerts by category
| Category | Count |
| brute-force | 1 |
| default-credentials | 1 |
| denial-of-service | 1 |
## MITRE ATT&CK techniques observed
| T1110.001 | Password Guessing | 1 |
| T1498 | Network Denial of Service | 1 |
```

Listing 4. Excerpt of a generated threat-analysis report.

REFERENCES

- [1] J. Lasquety-Reyes, “Global: Smart home—number of users 2018–2027,” Statista.
- [2] “What is a smart home and what are the benefits?,” Constellation.com.
- [3] “Smart home: Threats and countermeasures,” Rambus, 2017.
- [4] “A Study of Smart Home Environment and its Security Threats,” ResearchGate.
- [5] “Smart home: Threats and countermeasures,” Rambus, 2017.
- [6] K. Subramanian and W. Meng, “Threat Hunting Using Elastic Stack: An Evaluation,” IEEE SOLI, 2021.
- [7] H. Qu, D. Qu and J. Tong, “Virtual environment for smart home,” IEEE CYBER, 2015.
- [8] “What is an intrusion detection system?,” DNSstuff, 2019.
- [9] E. Hasson and L. Cheng, “Honeypot,” Imperva Learning Center.
- [10] A. Churcher et al., “An experimental analysis of attack classification using ML in IoT networks,” Sensors, 21(2):446, 2021.
- [11] K. Subramanian and W. Meng, “Threat Hunting Using Elastic Stack,” IEEE SOLI, 2021.
- [12] Banerjee et al., “Network traffic analysis based IoT botnet detection using honeynet data,” 2021.
- [13] S. Bistarelli, E. Bosimini and F. Santini, “A Report on the Security of Home Connections with IoT and Docker Honeypots,” CEUR-WS Vol-2597.

[14] M. Harikanth and P. Rajarajeswari, "Malicious event detection using ELK stack through CTI," *IJITEE*, 8(7):882–886, 2019.

[15] MITRE ATT&CK® Enterprise Matrix, The MITRE Corporation.

[16] J. Gamblin, "Mirai BotNet source code," GitHub, 2016.

[17] Elastic B.V., "Elastic Stack documentation," 2024.

[18] OISF, "Suricata User Guide," 2024.